

DOCUMENTO DE ENSAYO

Seguridad como Codigo: DevSecOps Spec-Driven sobre GitHub para Banca

Techno Week 8.0 — Banco de Costa Rica — 18 de mayo 2026

#	SECCION	DURACION	ACUMULADO	SLIDES
1	Apertura — El Hook	3:30	0:00–3:30	1–3
2	El Problema	5:30	3:30–9:00	4–8
3	La Tesis	3:00	9:00–12:00	9–11
4	Arquitectura del Pipeline	8:00	12:00–20:00	12–17
5	DEMO EN VIVO	17:00	20:00–37:00	18–19
6	Resultados e Impacto	2:00	37:00–39:00	20–23
7	GHAS + Estandarizacion ★	5:00	39:00–44:00	24–26
8	ardops.dev + CTA	2:00	44:00–46:00	27–28
9	Cierre Memorable	2:00	46:00–48:00	29–30
10	Preguntas	12:00	48:00–60:00	—

★ = seccion nueva (GHAS + estandarizacion organizacional)

CHECKPOINTS: Min 12 = fin Tesis | Min 20 = inicio DEMO | Min 37 = fin DEMO | Min 39 = inicio GHAS | Min 48 = preguntas

SECCION 1: APERTURA — EL HOOK

0:00 – 3:30 (3.5 min)

[SLIDE 1 — Portada] [MOVER: Centro, mirar al publico, NO a la pantalla]

[TONO: Serio, pausado, como contando una historia real]

El 14 de agosto del 2025, un atacante accedio a la API de una empresa financiera en Estados Unidos. No uso un exploit sofisticado. No rompio un firewall. Simplemente... hizo un request a un endpoint que no validaba quien estaba pidiendo los datos.

[PAUSA 3 seg]

En menos de 48 horas, los datos de 5.8 millones de personas estaban comprometidos. Nombres, numeros de seguro social, informacion crediticia. Todo.

[PAUSA 2 seg] [TONO: Conversacional]

Y saben que es lo mas absurdo? Esa vulnerabilidad — un endpoint sin validacion de identidad — la hubiera detectado un linter de OpenAPI en 0.3 segundos.

[PAUSA 2 seg] [MOVER: Paso hacia el publico] [TONO: Conviccion]

0.3 segundos contra 5.8 millones de victimas. Esa es la diferencia entre revisar la seguridad al final... y tenerla integrada desde el primer commit.

[SLIDE 2 — Nombre] [TONO: Calido, sonreir]

Buenas tardes. Mi nombre es [TU NOMBRE], soy DevOps Engineer, y hoy vamos a hablar de como convertir la seguridad en codigo. No en documentos de 200 paginas. No en auditorias que llegan tres meses despues. Codigo. Que se ejecuta, se versiona, y les dice en tiempo real si su API bancaria es segura o no.

[TONO: Ligeramente humoristico]

Y lo mejor: al final, cada uno va a tener acceso a un repositorio de GitHub donde todo esto funciona. Lo clonan, lo adaptan, y manana ya tienen un pipeline de seguridad automatizado. Sin excusas.

[SLIDE 3 — Que tan segura es tu API?]

Pero antes, necesito que entendamos juntos por que lo que hacemos hoy... no es suficiente.

SECCION 2: EL PROBLEMA

3:30 – 9:00 (5.5 min)

[SLIDE 4 — 84%]

Levanten la mano los que trabajan con APIs en su día a día.

[INTERACCION: Esperar. Sonreir.] Perfecto. Casi todos.

En 2025, el 84% de las empresas reportaron al menos un incidente de seguridad en APIs. Y ojo — esos son los que lo detectaron.

[SLIDE 5 — Solo el 13%]

Solo el 13% puede prevenir mas de la mitad de los ataques. El otro 87%? En modo 'ojala no nos pase'. Como poner un candado de bicicleta en la boveda del banco.

[SLIDE 6 — Espejo retrovisor] El modelo clasico:

Desarrollo construye. Despliegan. Y DESPUES llega seguridad. Pen test. 47 hallazgos. PDF de 200 paginas. El dev lo abre y dice: 'Mae, esto lo veo la proxima semana.' Esa semana nunca llega.

[SLIDE 7 — Costa Rica SUGEF] [TONO: Serio, para tomadores de decision]

En CR, CONASSIF reformo el Reglamento de Gestion TI en 2024. SUGEF exige controles verificables. Y en febrero 2026, la Asamblea aprobo en primer debate una ley que hace a los bancos responsables si no demuestran cumplimiento. Esto ya no es tecnico. Es legal.

[SLIDE 8 — Y si la spec fuera el contrato de seguridad?]

SECCION 3: LA TESIS

9:00 – 12:00 (3 min)

[SLIDE 9 — Seguridad comoCodigo] [MOVER: Centro] [TONO: Energetico — momento aha!]

La solucion no es mas auditores ni mas herramientas. Es cambiar CUANDO y COMO se aplica la seguridad. Que pasaria si cada PR ejecutara 6 validaciones automaticas? Sin que nadie lo pida?

[SLIDE 10 — La Spec genera todo]

Spec-Driven Security: tu OpenAPI no es solo docs. Es el contrato de seguridad. Genera: linting, tests de auth, ZAP, deps, secretos, compliance. Todo en GitHub Actions. Todo como codigo.

[SLIDE 11 — Por que Spec-Driven] Tres razones:

1) La spec es la fuente de verdad. 2) Es auditable — tu repo ES la evidencia para SUGEF. 3) Es reproducible — corre siempre igual.

[RESPIRAR]

CHECKPOINT 12:00 — Terminando tesis. Si atrasado, recortar un ejemplo.

SECCION 4: ARQUITECTURA DEL PIPELINE

12:00 – 20:00 (8 min)

[SLIDE 12 — Titulo] Ya entendemos el por que. Ahora el como.

[SLIDE 13 — Pipeline] [MOVER: Senalar pantalla izq a der]

Todo empieza con openapi.yaml. Cuando un dev hace PR: 6 etapas automaticas.

[SLIDE 14 — Spectral + Semgrep]

Etapas 1: Spectral — lint de la spec. Etapa 2: Semgrep — SAST, patrones peligrosos.

[SLIDE 15 — Gitleaks + npm audit]

Etapas 3: Gitleaks — secretos. (Broma: todos hemos sudado frio con un git push) Etapa 4: npm audit — CVEs.

[SLIDE 16 — OWASP ZAP + Compliance]

Etapas 5: ZAP — levanta API, la ataca con la spec. Etapa 6: reporte automatico en el PR.

[SLIDE 17 — Vista del PR]

Verde = seguro. Rojo = corregir. El reviewer no necesita ser experto en seguridad.

[TONO: Confiado] Y ahora... dejemos de hablar de teoria. Quiero que lo vean funcionando.

CHECKPOINT 20:00 — Empezando DEMO. Si atrasado, recortar 1 etapa.

SECCION 5: DEMO EN VIVO

20:00 – 37:00 (17 min)

[SLIDE 18 — DEMO EN VIVO] [MOVER: Caminar a la laptop]

TIEMPO	PASO	QUE HACER / DECIR
20:00–22:00	1. Mostrar repo	README, estructura, SECURITY.md. NO abrir archivos completos.
22:00–25:00	2. OpenAPI Spec	3 endpoints. Senalar security en cada uno. Formatos, limites.
25:00–27:00	3. Vulnerabilidad	[Tono conspirativo] Agregar endpoint sin auth. Commit + PR.
27:00–32:00	4. Pipeline	Ver 6 checks. Spectral ROJO. Semgrep ROJO. ZAP ROJO.
32:00–34:00	5. Reporte	Comentario automatico en PR. Semaforo visual.
34:00–37:00	6. Fix	Agregar security + auth. Push. Todo VERDE. Listo para merge.

PLAN B: Si tarda -> screenshots. Si falla -> 'Murphy's Law, por eso necesitamos pipelines' -> video backup. NUNCA 'esto funcionaba ayer'.

[Despues de la demo] '0.3 segundos vs 5.8 millones. Recuerdan?'

CHECKPOINT 37:00 — DEMO terminada. Volver al centro del escenario.

SECCION 6: RESULTADOS (comprimida)

37:00 – 39:00 (2 min)

[SLIDE 20 — Resultados] + [SLIDE 21 — Antes vs Despues]

Deteccion: de semanas a minutos. Cobertura: de 60% manual a 100% de la spec. Evidencia: de PDFs perdidos a trazabilidad en cada PR.

[SLIDE 22 — \$9.36 millones]

Costo promedio de un breach en banca en EE.UU. Implementar este pipeline cuesta: tiempo, herramientas open source, y voluntad.

[SLIDE 23 — Compliance automatico] [TONO: Para tomadores de decision]

Cada PR genera evidencia. Cada merge tiene trazabilidad. Cuando SUGEF llegue, la evidencia ya existe.

SECCION 7: GHAS + ESTANDARIZACION ★ NUEVA

39:00 – 44:00 (5 min)

★ Esta seccion es nueva — cumple con el requisito de gerencia sobre GitHub Advanced Security y estandarizacion organizacional.

[SLIDE 24 — GitHub Advanced Security] [TONO: Conectando con realidad enterprise]

Todo lo que mostre usa herramientas open source. Pero GitHub ofrece algo nativo que para un banco es game changer: GitHub Advanced Security. Tres componentes:

Primero: CodeQL. Motor de analisis semantico de GitHub. No solo busca patrones como Semgrep — entiende como los datos fluyen desde el input del usuario hasta la base de datos. Se configura con dos clicks o con un workflow de Actions.

Segundo: Secret Scanning con Push Protection. Clave para banca. En vez de detectar un secreto DESPUES del push, GitHub lo bloquea ANTES. El dev intenta hacer push con una API key y GitHub dice: No. Esto no sale de tu maquina. Prevencion, no solo deteccion.

Tercero: Dependabot. Monitorea dependencias 24/7. Crea PRs automaticos cuando hay un CVE. No hay que esperar a que alguien corra npm audit.

[SLIDE 25 — Open Source + GHAS: modelo hibrido]

Mi recomendacion para un banco: modelo hibrido. Las herramientas open source del pipeline — Spectral, ZAP — son el core para seguridad spec-driven. GHAS agrega la capa enterprise: CodeQL para analisis profundo, Push Protection para prevencion, Dependabot para monitoreo continuo. Secret Protection: \$19/commiter/mes. Code Security: \$30. Para repos publicos, muchas features son gratis.

[SLIDE 26 — De 1 repo a 200] [TONO: Dirigido a tomadores de decision]

La pregunta de gerencia: esto funciona para un repo, como lo aplico a los 200 del banco? Tres mecanismos:

Uno: Reusable Workflows. Definir el pipeline UNA vez en un repo central. Todos los equipos lo invocan con una sola linea. Si actualizas una regla, se propaga a todos.

Dos: Organization Rulesets. Desde la config de la org, reglas obligatorias: ningun PR se mergea a main sin que el workflow de seguridad pase. Ni un admin puede saltarselo sin quedar registrado.

Tres: Custom Properties. Clasificar repos por nivel de riesgo — alto, medio, bajo. Repos con datos de clientes: 6 etapas obligatorias. Repos internos: subconjunto.

Resultado: un estandar de seguridad organizacional, definido como codigo, aplicado automaticamente, auditado por GitHub. Exactamente lo que SUGEF y la nueva ley necesitan ver.

CHECKPOINT 44:00 — Terminando GHAS. Transicion a ardops.dev.

SECCIONES 8-9: ARDOPS.DEV + CIERRE

44:00 – 48:00 (4 min)

[SLIDE 27 — ardops.dev] + [SLIDE 28 — QR]

Todo lo que vimos hoy esta en ardops.dev. Repo, slides, guias. [QR] Saquenle foto ahora. [PAUSA 10 seg]

[SLIDE 29 — La seguridad no es un gate. Es un guardrail.]

[MOVER: Centro. Contacto visual. TONO: Pausado, con peso]

La seguridad no es un gate. No es una barrera al final. Es un guardrail. Esta ahí MIENTRAS construyes. Te guía. Te protege sin frenarte.

[PAUSA 3 seg]

Mi reto: la próxima vez que hagan un PR, preguntense: cuantos checks de seguridad se ejecutaron automáticamente? Si la respuesta es cero... ya saben por donde empezar.

[SLIDE 30 — Gracias / ardops.dev] [PAUSA 3 seg]

Gracias. El repo es publico, ardops.dev tiene todo. Estoy aquí para preguntas.

[MOVER: Quedarse en el centro. Tomar agua. No ir al podio.]

CHECKPOINT 48:00 — Abriendo preguntas. Si nadie pregunta:

'Una pregunta que siempre me hacen: esto no hace mas lento el desarrollo?' -> respuesta preparada. Planted questions: 1) APIs legacy sin spec? 2) Reemplaza equipo de seguridad? 3) Cuanto tiempo? 4) Como escala GHAS a muchos repos?

REFERENCIA RAPIDA

DATOS CLAVE:

- 84% de empresas tuvieron incidente en APIs (Traceable.ai 2025)
- Solo 13% puede prevenir mas del 50% de ataques a APIs
- \$9.36M costo promedio de breach en banca EE.UU. (2024, Statista/IBM)
- 5.8M registros en breach de 700Credit (ago 2025, API sin validacion)
- 3,322 data breaches en EE.UU. en 2025 — record historico (ITRC)
- 57% de orgs sufrieron breach por APIs en ultimos 2 anos
- CONASSIF 5-24: reforma Reglamento Gestion TI (julio 2024)
- Ley de responsabilidad bancaria: primer debate, febrero 2026
- GHAS: Secret Protection \$19/mes, Code Security \$30/mes por committer

PIPELINE (7 etapas con GHAS):

#	HERRAMIENTA	TIPO	QUE HACE
1	Spectral	Spec Lint	Reglas de seguridad en OpenAPI
2	Semgrep	SAST	Patrones de codigo peligrosos
3	Gitleaks	Secrets	API keys, passwords en codigo
4	npm audit	Deps	CVEs en dependencias
5	OWASP ZAP	DAST	Ataca la API con la spec
6	Custom Action	Report	Reporte compliance en PR
7	CodeQL (GHAS)	SAST+	Analisis semantico de flujo de datos

ESTANDARIZACION ORGANIZACIONAL:

- Reusable Workflows: definir UNA vez, invocar desde N repos con 1 linea
- Organization Rulesets: ningun PR se mergea sin pasar seguridad (obligatorio)
- Custom Properties: clasificar repos por riesgo (alto/medio/bajo)

FRASES DE EMERGENCIA:

- No sabes la respuesta:** 'Excelente pregunta. Me comparten su contacto? Les mando la respuesta esta semana.'
- Te interrumpen:** 'Entiendo. Permitame terminar y lo discutimos en preguntas.'
- Demo falla:** 'Murphy's Law. Por eso necesitamos pipelines automatizados.' -> video backup
- Te quedas en blanco:** Ir a la slide. Leer titulo. Tomar agua. 'Dejenme retomar...'

Preguntan por costo GHAS: '\$19 Secret Protection + \$30 Code Security por committer/mes.
Repos publicos: gratis.'

*Hablar al 70% de velocidad. Pausar despues de datos impactantes. Contacto visual 3 zonas. Manos fuera de bolsillos.
Agua en transiciones. Respirar. Sonreir.*